TrackOps — API Documentation

API Name: TrackOps REST API | **Version:** 1.0 (unversioned `/api` prefix) | **Owner:** Owner / Admin | **Base URL:** `http://localhost:4000/api` (also reachable as `http://localhost:3000/api` through the Vite dev proxy)

Source of truth: `server/index.ts` and the SQL files in `server/`. Last reviewed 29 Oct 2026.

10.1 Overview

The TrackOps API is an Express 5 + `pg` service backed by a local PostgreSQL database. It powers the Inventory, Sales, Purchases and Finances pages.

Conventions

- Requests and responses are JSON (`Content-Type: application/json`).
- Field names are camelCase in JSON; database columns are snake_case.
- Money is in Indian Rupees (₹), returned as JSON numbers rounded to 2 decimals. Quantities allow up to 3 decimals.
- Dates in filters and registers are `YYYY-MM-DD`. All day, month and range boundaries are calculated in **Asia/Kolkata**. Timestamps are ISO 8601 strings.
- Sale and purchase `:id` accepts either the numeric database id or the voucher number (`S-0001`, `P-0001`, case-insensitive).
- Sales, purchases and stock changes run in database transactions with row locks (`SELECT ... FOR UPDATE`), so a failed request leaves no partial data.
- Many responses include duplicate alias fields for front-end compatibility (e.g. `total` and `totalAmount`; `saleNo` and `number`; `date` and `recordDate`).

Error format. Every error returns an HTTP status with `{ "error": "message" }`.

Status	Used for
200 / 201	Success / created
400	Validation failed or malformed id or date
404	Record not found, or unknown route (`API route not found`)
409	Conflict: duplicate product, insufficient stock, or an edit/delete that would make stock negative

Status	Used for
500	Unexpected database or server error

Data model (tables). `products`, `stock_movements`, `sales` + `sale_items`, `purchases` + `purchase_items`, `finance_records`. Stock-changing endpoints write append-only rows to `stock_movements`. Sales and purchases are never copied into `finance_records`; registers and reports read them live.

10.2 Authentication

None. The API has no login, tokens or API keys (PRD decision: single admin user, no login in v1). The server listens on `0.0.0.0:4000` and CORS is enabled for all origins, so protection depends entirely on keeping the PC on a private network (the owner also uses Tailscale). Do not expose port 4000 to the public internet.

Configuration (`.env`): `DATABASE_URL` (PostgreSQL connection string) and `PORT` (default 4000).

10.3 Endpoints

Endpoint index

Method	Path	Purpose
GET	`/health`	Health check
GET	`/products`	List/search products
GET	`/products/:id`	Get one product
POST	`/products`	Create product
PUT	`/products/:id/stock`	Set exact stock
PATCH	`/products/:id/stock`	Adjust stock by a change
DELETE	`/products/:id`	Delete product
POST	`/sales`	Create sale
GET	`/sales`	List sales
GET	`/sales/:id`	Get sale with items
PUT	`/sales/:id`	Replace/edit sale

Method	Path	Purpose
DELETE	`/sales/:id`	Delete sale
POST	`/purchases`	Create purchase
GET	`/purchases`	List purchases
GET	`/purchases/:id`	Get purchase with items
PUT	`/purchases/:id`	Replace/edit purchase
DELETE	`/purchases/:id`	Delete purchase
POST	`/finances`	Add manual income/expense
GET	`/finances`	List manual records
DELETE	`/finances/:id`	Delete manual record
GET	`/registers/expenses`	Expense Register (money out)
GET	`/registers/income`	Income Register (money in)
GET	`/reports/summary`	Totals for a date range
GET	`/reports/pnl`	Profit & Loss with previous-period comparison
GET	`/reports/expenses`	Expense report by category
GET	`/reports/charts`	Time-series data for charts

`GET /health`

Description: Confirms the API is running.

Request Parameters: none

Sample Response:

```
{ "ok": true }
```

Error Codes: none.

`GET /products`

Description: Lists all products sorted by name, then size. Optional case-insensitive partial match on name.

Request Parameters:

Name	Type	Required	Description
search	string (query)	No	Matches `name` with ILIKE `%search%`

Sample Response:

```
[
  { "id": 1, "name": "Batman", "unitType": "A4", "stockQty": 10, "costPrice": 40, "sellPrice": 80, "imagePath": null }
]
```

Error Codes:

Code	Meaning	Notes
500	Failed to fetch products	Database error

`GET /products/:id`

Description: Returns one product (same shape as above).

Request Parameters:

Name	Type	Required	Description
id	integer (path)	Yes	Positive integer product id

Error Codes:

Code	Meaning	Notes
400	Invalid product ID	Not a positive integer
404	Product with id N not found	
500	Failed to fetch product	

`POST /products`

Description: Creates a product. If `stockQty` is above 0, an `opening` stock movement is written in the same transaction. A product is unique by name + size, ignoring case.

Request Parameters:

Name	Type	Required	Description
name	string	Yes	Trimmed, 1–100 characters
unitType	string	Yes	One of `A4`, `A3`, `A5`, `Polaroid`
stockQty	number	No	0 or more; defaults to 0
costPrice	number	Yes	0 or more
sellPrice	number	Yes	0 or more

Sample Request:

```
{ "name": "Batman", "unitType": "A4", "stockQty": 10, "costPrice": 40,
"sellPrice": 80 }
```

Sample Response (201):

```
{ "id": 1, "name": "Batman", "unitType": "A4", "stockQty": 10, "costPrice":
40, "sellPrice": 80, "imagePath": null }
```

Error Codes:

Code	Meaning	Notes
400	Validation error	Name missing or over 100 chars; bad `unitType`; negative or non-numeric numbers
409	This poster already exists in this size	Duplicate name + size
500	Could not add product	

`PUT /products/:id/stock`

Description: Sets stock to an exact quantity. The difference is logged as an `adjustment` movement. If the new quantity equals the current one, nothing is written and the unchanged product is returned.

Request Parameters:

Name	Type	Required	Description
id	integer (path)	Yes	Product id

Name	Type	Required	Description
stockQty	number	Yes	New stock, 0 or more
reason	string	No	Stored on the movement; defaults to `Correction`. The UI offers Correction, Wastage, Miscellaneous (not enforced by the API)
note	string	No	Up to 200 characters

Sample Request:

```
{ "stockQty": 7, "reason": "Wastage", "note": "Water damage" }
```

Sample Response (200): the updated product object.

Error Codes:

Code	Meaning	Notes
400	Invalid product id / stockQty must be 0 or more / note too long	
404	Product not found	
500	Could not update stock	

`PATCH /products/:id/stock`

Description: Adds or removes a quantity from current stock and logs an `adjustment` movement. The resulting stock cannot be negative.

Request Parameters:

Name	Type	Required	Description
id	integer (path)	Yes	Product id
changeQty	number	Yes	Positive or negative change. `delta` or `change` are accepted as aliases
reason	string	No	Defaults to `Correction`
note	string	No	Up to 200 characters

Sample Request:

```
{ "changeQty": -2, "reason": "Damage" }
```

Error Codes:

Code	Meaning	Notes
400	Invalid id / changeQty must be a valid number / Resulting stock cannot be negative	
404	Product not found	
500	Could not update stock	

`DELETE /products/:id`

Description: Deletes the product. Past sale items, purchase items and stock movements are kept: their `product_id` becomes null while product name, size and amounts remain. The name + size can be re-added later.

Sample Response (200):

```
{ "deleted": true, "id": 1, "name": "Batman" }
```

Error Codes:

Code	Meaning	Notes
400	Invalid product id	
404	Product not found	
500	Could not delete product	

`POST /sales`

Description: Records a sale. Prices are always taken from the product's current `sellPrice` and cost from its `costPrice` (any price sent by the client is ignored); the cost is locked on the sale item for COGS. Duplicate `productId` lines are merged. Stock is reduced and one `sale` movement is written per product. Voucher numbers are generated as `S-0001`, `S-0002`, …

Request Parameters:

Name	Type	Required	Description
paymentMethod	string	Yes	`Cash` or `UPI`
soldAt	string (ISO date-time)	No	Defaults to now
items	array	Yes	Non-empty list of `{ productId, qty }`
items[].productId	integer	Yes	Existing product
items[].qty	number	Yes	Greater than 0

Sample Request:

```
{ "paymentMethod": "UPI", "items": [ { "productId": 1, "qty": 2 }, { "productId": 3, "qty": 1 } ] }
```

Sample Response (201):

```
{
  "id": 12, "saleNo": "S-0012", "number": "S-0012",
  "paymentMethod": "UPI", "totalAmount": 240, "total": 240,
  "soldAt": "2026-10-29T08:15:00.000Z", "createdAt": "2026-10-29T08:15:00.000Z",
  "items": [
    { "id": 30, "saleId": 12, "productId": 1, "productName": "Batman", "unitType": "A4",
      "qty": 2, "unitPrice": 80, "price": 80, "unitCost": 40, "cost": 40, "lineTotal": 160 }
  ]
}
```

Error Codes:

Code	Meaning	Notes
400	Invalid payment method / empty items / bad productId or qty	
404	Product with id N not found	
409	Not enough stock for …	Body also has `availableStock`
500	Could not save sale	

`GET /sales`

Description: Lists sales newest first with an item count. No line items; use `GET /sales/:id` for those.

Request Parameters:

Name	Type	Required	Description
from	date (query)	No	Inclusive start, `YYYY-MM-DD`, Asia/Kolkata. Ignored if malformed
to	date (query)	No	Inclusive end
search	string (query)	No	Matches sale number or any item's product name

Sample Response:

```
[ { "id": 12, "saleNo": "S-0012", "number": "S-0012", "paymentMethod": "UPI",
"totalAmount": 240, "total": 240,
    "date": "2026-10-29", "soldAt": "2026-10-29T08:15:00.000Z", "createdAt":
"2026-10-29T08:15:00.000Z", "itemCount": 2 } ]
```

Error Codes: 500 Failed to fetch sales.

`GET /sales/:id`

Description: Returns one sale with its items (same shape as the POST response).

Error Codes:

Code	Meaning	Notes
400	Invalid sale ID format	Must be digits or `S-<digits>`

Code	Meaning	Notes
404	Sale not found	
500	Failed to fetch sale	

`PUT /sales/:id`

Description: Replaces a sale's payment method, date and items. Stock is reconciled by the net difference per product and compensating `sale` movements (note `Edited S-XXXX`) are written. Items already on the sale keep their original `unitPrice` and `unitCost`; newly added items use current catalogue prices. Available stock for a line is current stock plus that sale's old quantity.

Request Parameters: same body as `POST /sales`.

Sample Response (200): the updated sale with items.

Error Codes:

Code	Meaning	Notes
400	Invalid id / payment method / items	
404	Sale or product not found	
409	Not enough stock for …	
500	Could not update sale	

`DELETE /sales/:id`

Description: Deletes the sale and its items and returns the sold quantities to stock, with compensating movements (note `Deleted S-XXXX`).

Sample Response (200):

```
{ "deleted": true, "id": 12, "saleNo": "S-0012" }
```

Error Codes: 400 Invalid sale ID format · 404 Sale not found · 500 Could not delete sale.

`POST /purchases`

Description: Records a stock purchase. Stock increases and one `purchase` movement is written per product. Each product's `costPrice` is set to the purchase cost (latest-cost method) when the cost is above 0. Duplicate product lines are merged: quantities add,

and the last cost given is used. Voucher numbers are `P-0001`, `P-0002`, …

Request Parameters:

Name	Type	Required	Description
paymentMethod	string	Yes	`Cash` or `UPI`
note	string	No	Free text
purchasedAt	string (ISO date-time)	No	Defaults to now
items	array	Yes	Non-empty list of `{ productId, qty, cost }`
items[].productId	integer	Yes	Existing product
items[].qty	number	Yes	Greater than 0
items[].cost	number	Yes	Unit cost, 0 or more (note: the field is `cost`, not `unitCost`)

Sample Request:

```
{ "paymentMethod": "Cash", "note": "Printer batch", "items": [ { "productId": 1, "qty": 20, "cost": 38 } ] }
```

Sample Response (201):

```
{
  "id": 5, "purchaseNo": "P-0005", "number": "P-0005", "paymentMethod": "Cash", "note": "Printer batch",
  "totalCost": 760, "total": 760, "purchasedAt": "2026-10-29T08:20:00.000Z", "createdAt": "2026-10-29T08:20:00.000Z",
  "items": [ { "id": 9, "purchaseId": 5, "productId": 1, "productName": "Batman", "unitType": "A4",
               "qty": 20, "unitCost": 38, "cost": 38, "totalCost": 760, "lineTotal": 760 } ]
}
```

Error Codes:

Code	Meaning	Notes
400	Invalid payment method / empty items / bad productId, qty or cost	

Code	Meaning	Notes
404	Product with id N not found	
500	Could not save purchase	

`GET /purchases`

Description: Lists purchases newest first with an item count.

Request Parameters: `from`, `to`, `search` — same behaviour as `GET /sales` (search matches purchase number or product name).

Sample Response: array of `{ id, purchaseNo, number, paymentMethod, note, totalCost, total, date, purchasedAt, createdAt, itemCount }`.

Error Codes: 500 Failed to fetch purchases.

`GET /purchases/:id`

Description: Returns one purchase with items. `:id` is a numeric id or `P-<digits>`.

Error Codes: 400 Invalid purchase ID format · 404 Purchase not found · 500 Failed to fetch purchase.

`PUT /purchases/:id`

Description: Replaces a purchase. Stock is adjusted by the net difference per product (compensating `purchase` movements, note `Edited P-XXXX`). If this was a product's most recent purchase, its `costPrice` is reset to the latest purchase item cost after the edit.

Request Parameters: same body as `POST /purchases`.

Error Codes:

Code	Meaning	Notes
400	Validation errors	As for POST
404	Purchase or product not found	
409	Cannot edit: <name> (<size>) stock would go below 0 (only N in stock)	Part of the stock has already been sold
500	Could not update purchase	

`DELETE /purchases/:id`

Description: Deletes the purchase and removes its quantities from stock. For each affected product whose latest purchase this was, `costPrice` reverts to the next most recent purchase cost (left unchanged if none remain).

Sample Response (200):

```
{ "deleted": true, "id": 5, "purchaseNo": "P-0005" }
```

Error Codes:

Code	Meaning	Notes
400	Invalid purchase ID format	
404	Purchase not found	
409	Cannot delete: <name> (<size>) has already been sold, stock would go below 0	
500	Could not delete purchase	

`POST /finances`

Description: Adds a manual income or expense (overheads and other income only). Sales and purchases must never be added here.

Request Parameters:

Name	Type	Required	Description
type	string	Yes	`income` or `expense`
category	string	Yes	Expense: Printing, Packaging, Transport, Rent, Utilities, Marketing, Other. Income: Other income
amount	number	Yes	Greater than 0 (rounded to 2 decimals)
paymentMethod	string	Yes	`Cash` or `UPI`
recordDate	string	No	`YYYY-MM-DD`; defaults to today in Asia/Kolkata
note	string	No	Up to 200 characters

Sample Request:

```
{ "type": "expense", "category": "Transport", "amount": 150, "paymentMethod":
"Cash", "recordDate": "2026-10-29", "note": "Delivery" }
```

Sample Response (201):

```
{ "id": 7, "type": "expense", "category": "Transport", "amount": 150,
"recordDate": "2026-10-29", "date": "2026-10-29",
  "paymentMethod": "Cash", "note": "Delivery", "createdAt": "2026-10-
29T08:30:00.000Z" }
```

Error Codes:

Code	Meaning	Notes
400	Invalid type, category, amount, payment method, or note too long	Error text lists allowed categories
500	Failed to create finance record	

`GET /finances`

Description: Lists manual finance records, newest first. Does not include sales or purchases (use the registers for that).

Request Parameters:

Name	Type	Required	Description
from	date (query)	No	Inclusive `record_date` start
to	date (query)	No	Inclusive end
type	string (query)	No	`income` or `expense`

Error Codes: 500 Failed to fetch finances.

`DELETE /finances/:id`

Description: Deletes a manual record.

Sample Response (200):

```
{ "deleted": true, "id": 7 }
```

Error Codes: 400 Invalid finance record ID · 404 Finance record not found · 500 Failed to delete finance record.

`GET /registers/expenses`

Description: Money-out register combining manual expenses and stock purchases, read live (nothing is duplicated). Sorted by date, then creation time, newest first.

Request Parameters:

Name	Type	Required	Description
from	date	Yes	`YYYY-MM-DD`
to	date	Yes	`YYYY-MM-DD`; must not be before `from`
source	string	No	`all` (default), `expense` or `purchase`
search	string	No	Filters category, description or reference
limit	integer	No	Default 100; must be 1 or more
offset	integer	No	Default 0

Sample Response:

```
{
  "entries": [
    { "id": 5, "source": "purchase", "date": "2026-10-29", "reference": "P-
0005", "category": "Stock purchase",
      "description": "Batman A4 x20", "amount": 760, "paymentMethod": "Cash",
"deletable": false },
    { "id": 7, "source": "expense", "date": "2026-10-29", "reference": null,
"category": "Transport",
      "description": "Delivery", "amount": 150, "paymentMethod": "Cash",
"deletable": true }
  ],
  "totals": { "expensesTotal": 150, "purchasesTotal": 760, "moneyOutTotal":
910, "count": 2 },
  "expensesTotal": 150, "purchasesTotal": 760, "moneyOutTotal": 910, "count":
2
}
```

Error Codes: 400 Valid 'from' and 'to' dates are required / Invalid date range / Invalid source filter / Invalid limit or offset · 500 Failed to fetch expenses register.

Totals cover the whole filtered range, not just the returned page.

`GET /registers/income`

Description: Money-in register combining sales revenue and manual other income.

Request Parameters: same as the Expense Register, except `source` is `all` (default), `sale` or `other`.

Sample Response: `entries` use `source` = `sale` (`reference` S-XXXX, category `Sales revenue` , `deletable` false) or `other` (category `Other income` , `deletable` true). `totals` = `{ salesTotal, otherIncomeTotal, moneyInTotal, count }` , also repeated at the top level.

Error Codes: as above; 500 Failed to fetch income register.

`GET /reports/summary`

Description: Headline figures for a date range.

Request Parameters: `from` , `to` (required, `YYYY-MM-DD`).

Sample Response:

```
{ "salesRevenue": 5000, "otherIncome": 200, "moneyIn": 5200, "expenses": 600,
"purchases": 2500,
  "moneyOut": 3100, "cogs": 2400, "grossProfit": 2600, "netProfit": 2200 }
```

Rules: `moneyIn` = sales + other income; `moneyOut` = expenses + purchases; `cogs` = sum of sale item qty × locked unit cost; `grossProfit` = sales − COGS; `netProfit` = gross profit + other income − expenses. Purchases are **not** deducted from profit.

Error Codes: 400 invalid or missing dates · 500 Could not fetch summary report.

`GET /reports/pnl`

Description: Profit & Loss for the range plus the immediately preceding range of identical length.

Request Parameters: `from` , `to` (required).

Response fields: all `summary` fields at the top level, plus `summary` , `previousSummary` , `expensesByCategory` (every expense category with `amount` , `previousAmount` , `changeAmount` , `changePercent`), `marginPercent` , `previousMarginPercent` , `changes` (for salesRevenue, cogs, grossProfit, otherIncome, expenses, netProfit, marginPercent, moneyIn, moneyOut, purchases — each `{ current, previous, changeAmount, changePercent }`), `currentRange` and `previousRange` (`{ from, to }`). `changePercent` is `null` when the previous value is 0; `marginPercent` is net profit ÷ sales revenue × 100 (0 if no sales).

Error Codes: 400 invalid dates · 500 Could not fetch PnL report.

`GET /reports/expenses`

Description: Operating expenses (manual entries only, no purchases) for the range.

Request Parameters: `from`, `to` (required).

Sample Response:

```
{ "totalExpenses": 600, "totalsByCategory": [ { "category": "Rent", "amount": 400 } ], "entries": [ /* finance records */ ] }
```

Error Codes: 400 invalid dates · 500 Could not fetch expenses report.

`GET /reports/charts`

Description: Gap-free time series for charts. The visual Reports page that consumed it is parked for the next version, but the endpoint is still live.

Request Parameters:

Name	Type	Required	Description
from	date	Yes	`YYYY-MM-DD`
to	date	Yes	`YYYY-MM-DD`
group	string	No	`day` or `month`; if omitted, `day` for ranges of 31 days or fewer, otherwise `month`

Response fields: `group`, `incomeVsExpenses` (`{ period, moneyIn, expenses }`), `salesOverTime` (`{ period, amount }`), `purchasesOverTime` (`{ period, amount }`), `profitTrend` (`{ period, netProfit }`), `topSelling` (top-selling products), and `stockLevels` (`{ name, unitType, qty, status }` where status is `out` at 0 or less, `low` at 10 or less, otherwise `ok`). `period` is `YYYY-MM-DD` or `YYYY-MM`.

Error Codes: 400 invalid dates · 500 Could not fetch charts report.

10.4 Rate Limits

None. There is no throttling, request-size policy beyond Express defaults, or maximum on `limit` for the registers. This is acceptable for a single-user, local deployment.

10.5 Versioning & Deprecation Policy

The API is unversioned: all routes sit under `/api` and change with the application. Because the only client is the bundled front end in this repository, endpoints and front end are changed together; there is no deprecation period. If a second client is ever added, introduce a `/api/v1` prefix first.

Known limitations and points to confirm

- No authentication or authorisation (see 10.2).
- There is no endpoint that returns the `stock_movements` log; movements are written by the API but not currently readable through it.
- The `date` field in `GET /sales` and `GET /purchases` lists is the UTC calendar date of the timestamp, whereas the `from` / `to` filters use Asia/Kolkata. Entries near midnight IST can therefore show a date one day off from the range they were filtered by.
- `reason` on stock adjustments is free text; the Correction / Wastage / Miscellaneous list is enforced only in the UI.
- `server/sales_purchases_schema.sql` is an older schema (columns `number`, `total`, `created_at`); the running API uses the newer `server/sales_purchases.sql` (`sale_no`, `total_amount`, `sold_at`, `purchase_no`, `total_cost`, `purchased_at`). Use the newer file when setting up a database.
- `GET /reports/charts` returns `topSelling` but its item shape was not documented here; read the handler in `server/index.ts` before relying on it.
- The shown sample numbers are illustrative.